САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа № 6

Мониторинг ресурсов и микросервисов

Выполнил:

Корольков Артем

БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Настроить систему мониторинга в Grafana. Отобразить использование CPU, оперативной памяти, диска и сети, доступность микросервисов и журналы контейнеров.

Ход работы

1 Настройка мониторинга

На виртуальной машине настроил отдельный Docker Compose для Grafana, Prometheus, Loki, Alloy, Node Exporter и Blackbox Exporter. Мониторинг подключил к сети приложения.

ITMO-ACS-Backend-2026-A > БР1.2 > Корольков Артем > labs > lab6 > docker-compose.yml

```yaml
name: restaurant-monitoring

services:
  prometheus:
    image: prom/prometheus:v3.3.0
    restart: unless-stopped
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.retention.time=7d
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    extra_hosts:
      - 'host.docker.internal:host-gateway'
    networks: [monitoring]

  node-exporter:
    image: prom/node-exporter:v1.9.1
    restart: unless-stopped
    network_mode: host
    command:
      - --path.rootfs=/host
    volumes:
      - /:/host:ro,rslave

  blackbox-exporter:
    image: prom/blackbox-exporter:v0.26.0
    restart: unless-stopped
    command:
      - --config.file=/etc/blackbox.yml
```

Рисунок 1 — Фрагмент Compose для мониторинга

2 Сбор метрик и проверка сервисов

Настроил Prometheus для получения метрик виртуальной машины и проверок четырёх сервисов через Blackbox Exporter. Для каждого сервиса указал адрес маршрута health.

ITMO-ACS-Backend-2026-A > БР1.2 > Корольков Артем > labs > lab6 > ! prometheus.yml

```yaml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: node
    static_configs:
      - targets: ['host.docker.internal:9100']

  - job_name: services
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets:
          - http://api-gateway:3010/health
          - http://auth-service:3001/health
          - http://restaurant-service:3002/health
          - http://reservation-service:3003/health
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: blackbox-exporter:9115

```

Рисунок 2 — Цели Prometheus для метрик и проверок доступности

3 Сбор журналов

Настроил Alloy для сбора журналов контейнеров Docker и передачи их в Loki. В Grafana добавил панель для просмотра логов.

ITMO-ACS-Backend-2026-A > БР1.2 > Корольков Артем > labs > lab6 > config.alloy

```
discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"
}

discovery.relabel "containers" {
  targets = discovery.docker.containers.targets

  rule {
    source_labels = ["__meta_docker_container_name"]
    regex = "/(.*)"
    target_label = "container"
  }
}

loki.source.docker "containers" {
  host = "unix:///var/run/docker.sock"
  targets = discovery.relabel.containers.output
  labels = { job = "docker" }
  forward_to = [loki.write.local.receiver]
}

loki.write "local" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}

```

Рисунок 3 — Настройка сбора журналов контейнеров

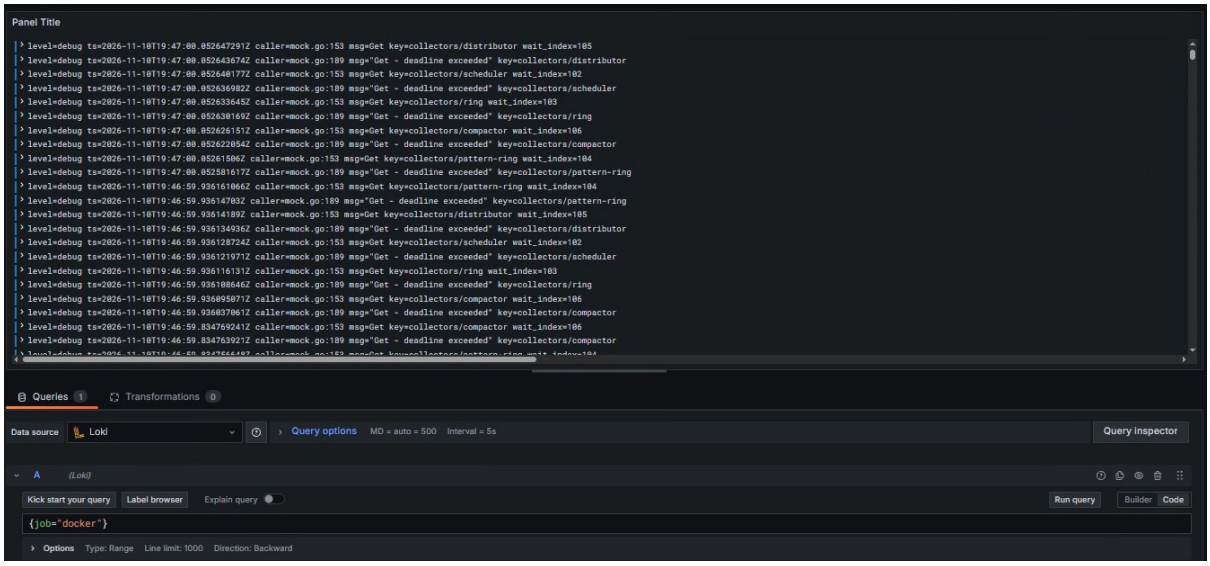


Рисунок 4 — Просмотр журналов в Grafana

4 Панели ресурсов

В Grafana добавил графики загрузки процессора, оперативной памяти и занятого места на диске. Для каждого графика использовал метрики Node Exporter.



Рисунок 5 — График загрузки CPU



Рисунок 6 — График использования оперативной памяти

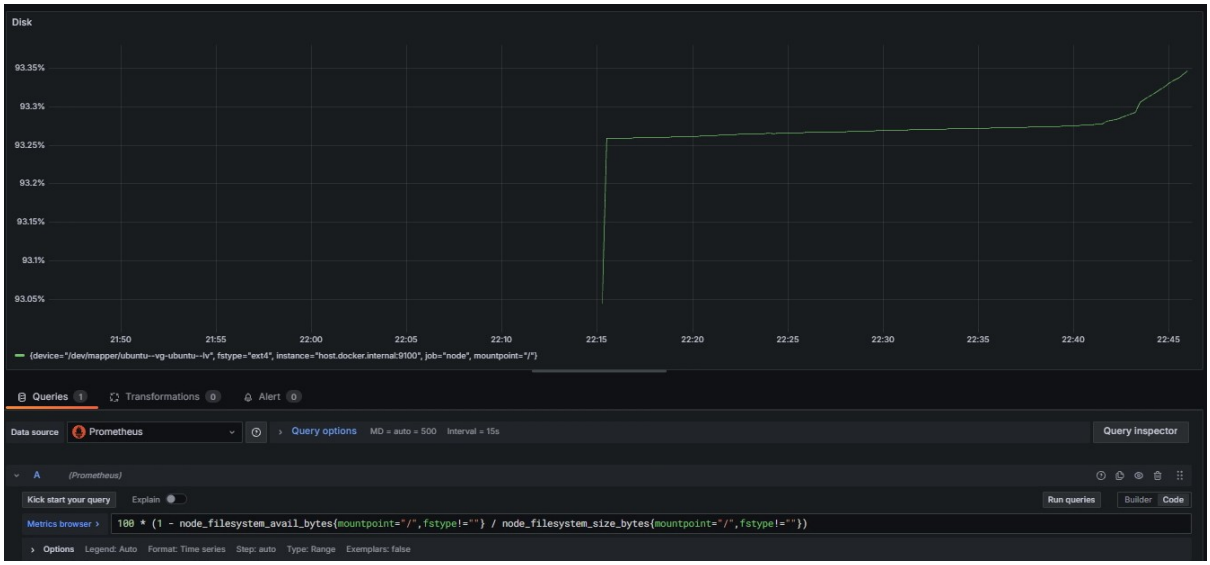


Рисунок 7 — График использования диска

5 Сеть и доступность

Добавил график сетевого трафика и панель состояний четырёх микросервисов. Доступные сервисы отображаются в состоянии UP.



Рисунок 8 — График сетевого трафика

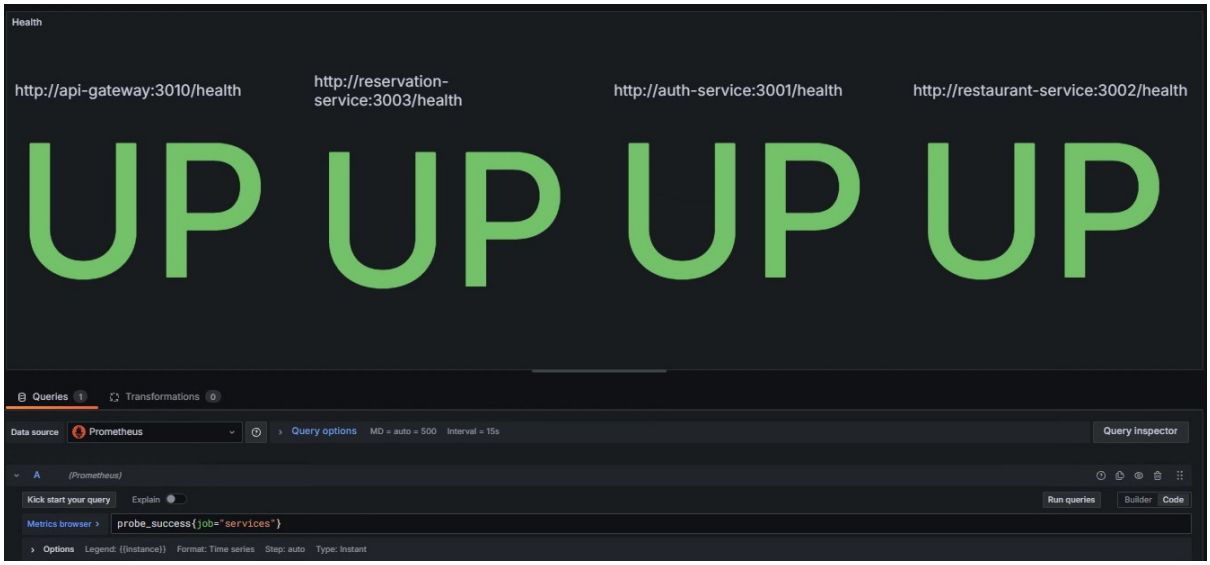


Рисунок 9 — Проверка доступности четырёх сервисов

6 Проверка при остановке сервиса

Временно остановил restaurant-service и проверил изменение его состояния на DOWN. После повторного запуска сервиса панель снова показала UP.

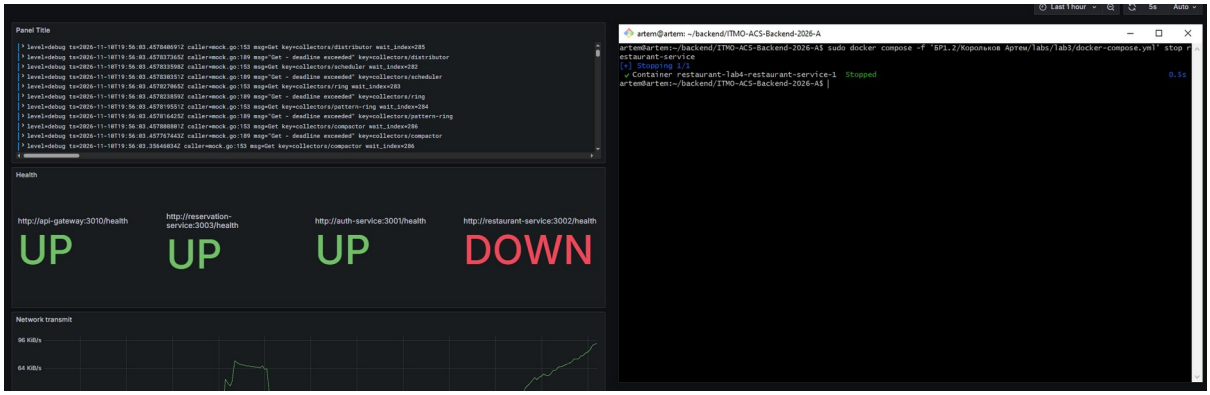


Рисунок 10 — Состояние сервиса после остановки

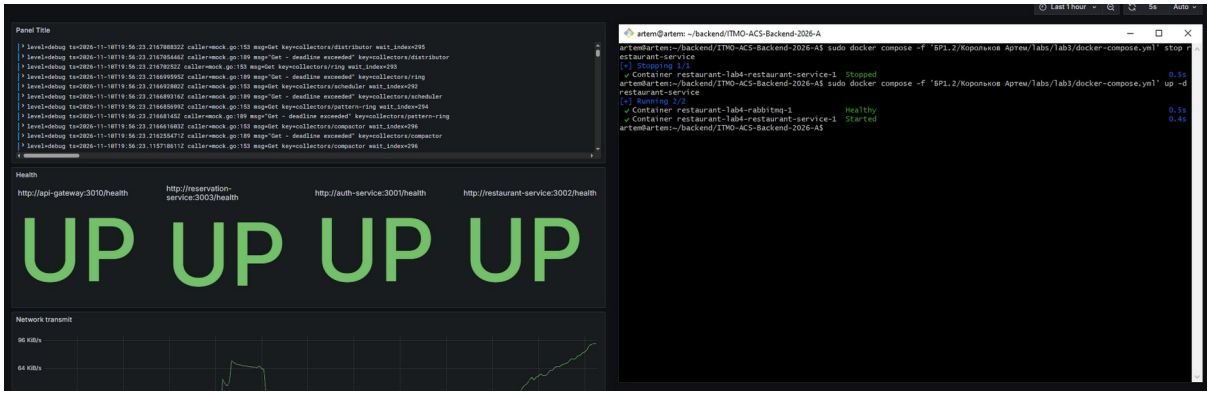


Рисунок 11 — Состояние сервисов после восстановления

7 Почтовое оповещение

Создал правило Grafana для проверки значения probe_success. При остановке сервиса получил на почту письмо о его недоступности.

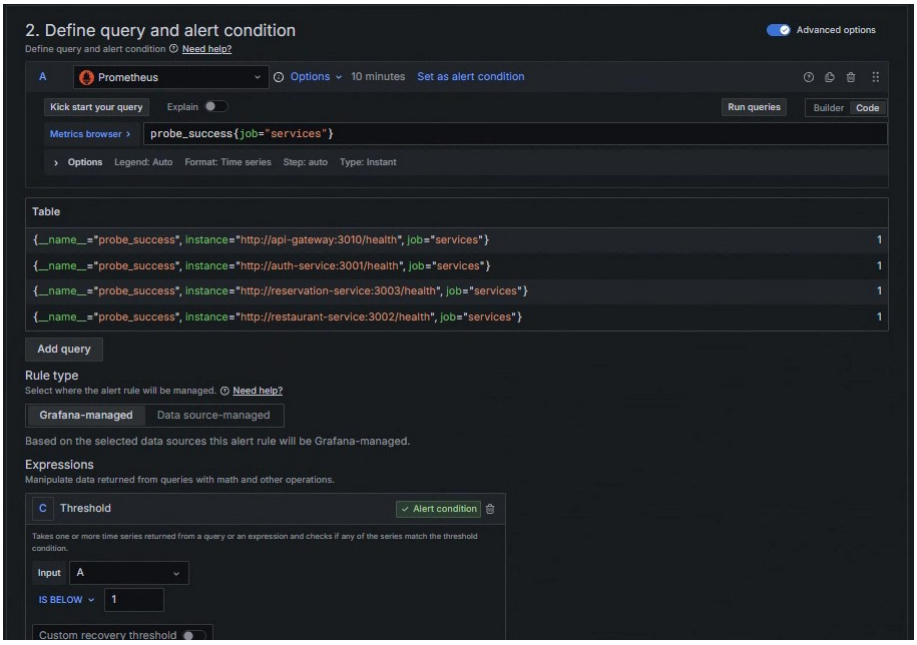


Рисунок 12 — Настройка правила проверки доступности

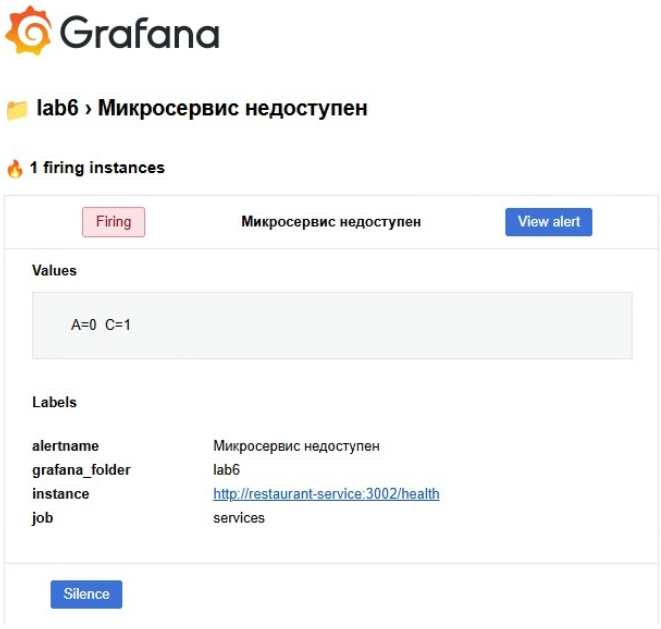


Рисунок 13 — Письмо Grafana о недоступности сервиса

8 Облачный Sentry

Подключил Sentry к API gateway для отправки ошибок подключения к другим сервисам. Проверил результат: в облачном проекте появилось событие TypeError: fetch failed.

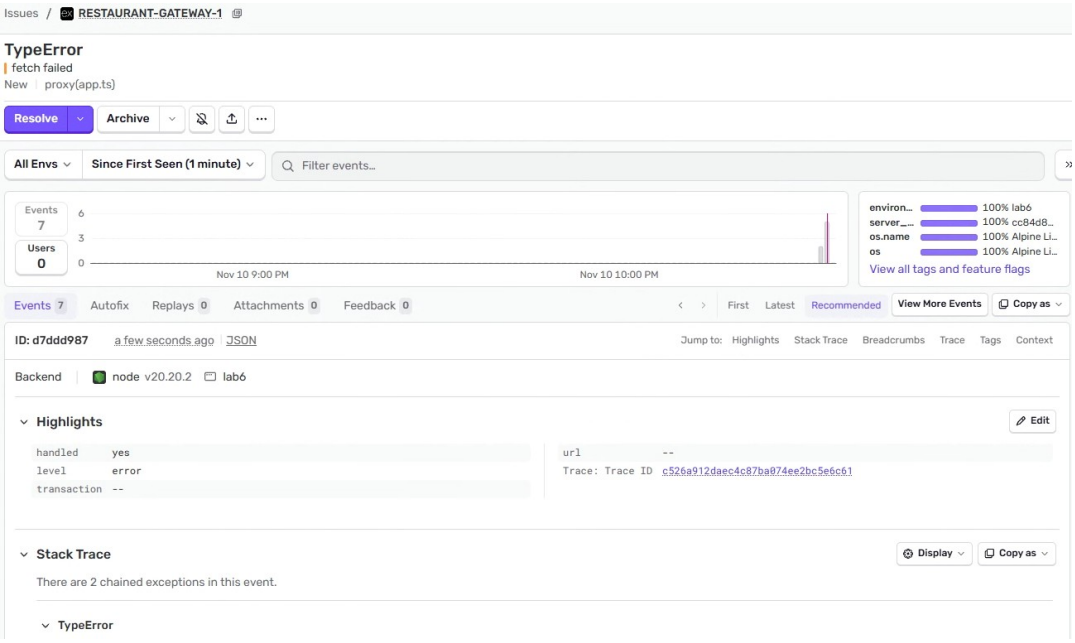


Рисунок 14 — Событие ошибки API gateway в Sentry

Вывод

Настроил мониторинг ресурсов виртуальной машины, доступности микросервисов и журналов контейнеров в Grafana. Проверил изменение состояния сервиса, получение письма и отправку ошибки gateway в облачный Sentry.